Отчет
о выполнении лабораторной работы № 6
Глубокие рекуррентные нейронные сети
Вариант 13

Выполнил студент группы ЗФИбд-01-24: Князев А. М.

Проверил: Леонов С. С.

Москва, 2026

Цель работы

изучение рекуррентных сетей для анализа текстовых последовательно
подготовка рекуррентной модели прогноза временных рядов.

Основные этапы работы

1. Построить двунаправленную LSTM-сеть для IMDB.
2. Сравнить результат с одномерной сверточной сетью.
3. Подготовить модель для прогноза температуры Jena Climate.

Ход выполнения и результаты

Таблица 1 — Архитектуры варианта

Этап | Параметры

IMDB BiLSTM | embedding 64, 2 слоя LSTM 32-32, bidirectional

Conv1D baseline | 3 одномерных сверточных слоя

Jena Climate | 3 bidirectional LSTM слоя 64-64-64, dropout 0.3

Таблица 2 — Результаты

Модель | Результат

BiLSTM variant | test_loss = 1.0329, test_acc = 0.7556

Conv1D baseline | test_loss = 1.2562, test_acc = 0.7667

Jena Climate model | 239 745 параметров

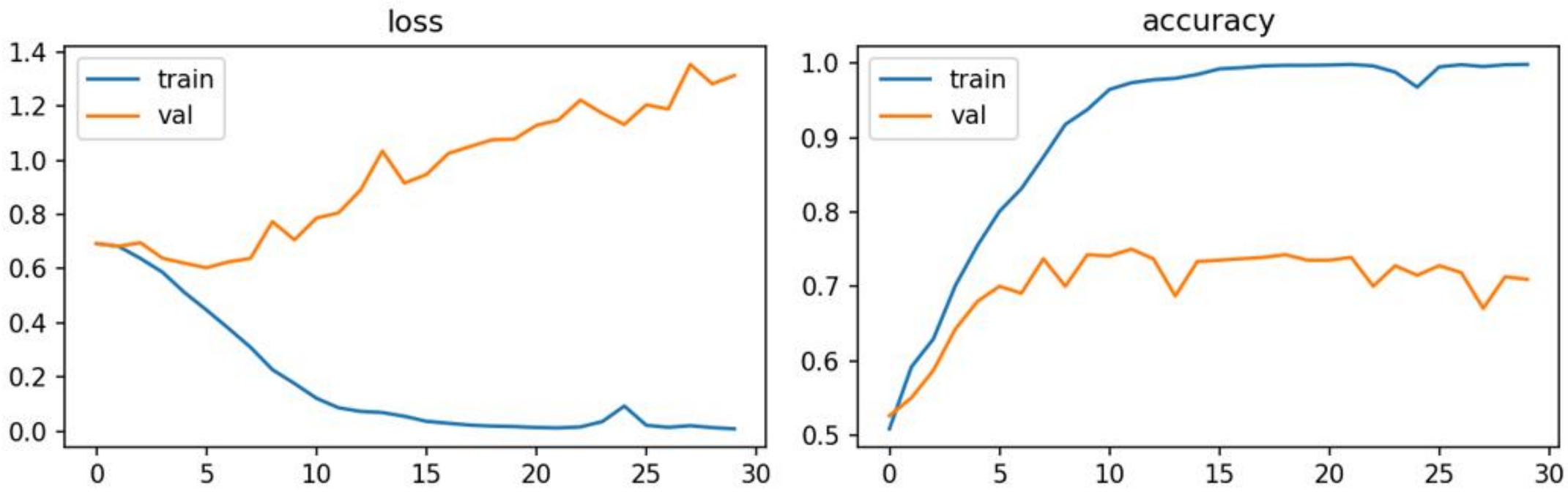


Рис. 1 — Графики loss и accuracy для BiLSTM

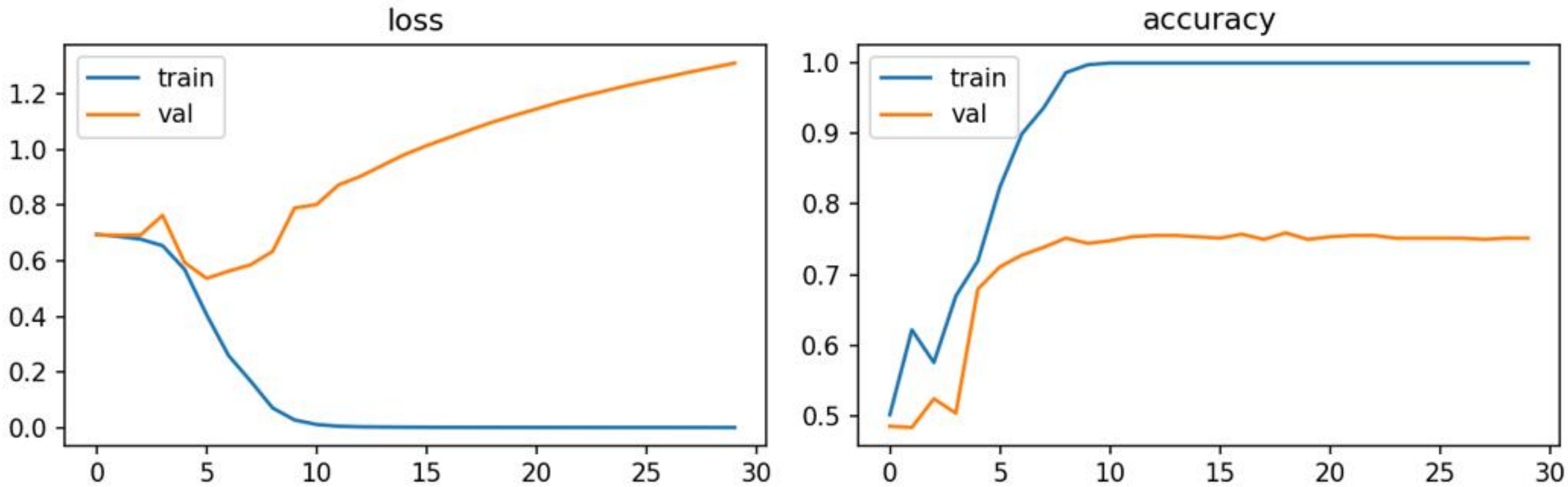


Рис. 2 — Графики loss и accuracy для Conv1D baseline

Вывод

Двунаправленная LSTM и одномерная сверточная сеть обучены на зада
IMDB. Обе модели дают рабочее качество, при этом Conv1D baseline
оказался немного точнее в выполненном прогоне. Для Jena Climate
подготовлена рекуррентная архитектура варианта 13.

Фрагменты кода из main.ipynb

```
from __future__ import annotations


import random

import re

import zipfile

from collections import Counter

from pathlib import Path
```

```
import matplotlib.pyplot as plt

import numpy as np

import torch

import torch.nn as nn

from torch.nn.utils.rnn import pad_sequence

from torch.utils.data import DataLoader, Dataset, TensorDataset, random_split
```

```


DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

SEED = 13

random.seed(SEED)

np.random.seed(SEED)

torch.manual_seed(SEED)
```

```


TOKEN_RE = re.compile(r"[a-zA-Z']+")
```

```


def tokenize(text: str) -> list[str]:

    return TOKEN_RE.findall(text.lower())
```

```


def read_imdb(zip_path: Path, max_docs_per_class: int = 900) -> tuple[list[str], list[int]]:

    texts: list[str] = []

    labels: list[int] = []

    with zipfile.ZipFile(zip_path) as archive:

        for split in ["train", "test"]:

            for label_name, label in [("neg", 0), ("pos", 1)]:

                prefix = f"aclImdb/{split}/{label_name}/"

                names = [n for n in archive.namelist() if n.startswith(prefix) and n.endswith(".txt")]

                for name in names[:max_docs_per_class]:

                    texts.append(archive.read(name).decode("utf-8", errors="ignore"))

                    labels.append(label)

    return texts, labels
```

```
class TextDataset(Dataset):
    def __init__(self, texts: list[str], labels: list[int], vocab: dict[str, int], max_len: int = 180):
        self.labels = labels
        self.samples = []
        for text in texts:
            ids = [vocab.get(token, 1) for token in tokenize(text)[:max_len]]
            self.samples.append(torch.tensor(ids or [1], dtype=torch.long))

    def __len__(self) -> int:
        return len(self.samples)

    def __getitem__(self, index: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.samples[index], torch.tensor(float(self.labels[index]), dtype=torch.float32)


def collate_batch(batch: list[tuple[torch.Tensor, torch.Tensor]]) -> tuple[torch.Tensor, torch.Tensor]:
    texts, labels = zip(*batch)
    return pad_sequence(texts, batch_first=True, padding_value=0), torch.stack(labels)


class BiLstmClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.rnn = nn.LSTM(
            input_size=embed_dim,
            hidden_size=32,
            num_layers=2,
            batch_first=True,
            bidirectional=True,
            dropout=0.2,
        )
        self.output = nn.Linear(64, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        embedded = self.embedding(x)
        _, (hidden, _) = self.rnn(embedded)
        last_forward = hidden[-2]
        last_backward = hidden[-1]
        return self.output(torch.cat([last_forward, last_backward], dim=1)).squeeze(1)


class ConvTextClassifier(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = 64):
```

```
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.features = nn.Sequential(
            nn.Conv1d(embed_dim, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Conv1d(64, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.MaxPool1d(2),
            nn.Conv1d(64, 64, kernel_size=5, padding=2),
            nn.ReLU(),
            nn.AdaptiveMaxPool1d(1),
        )
        self.output = nn.Linear(64, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        embedded = self.embedding(x).transpose(1, 2)
        features = self.features(embedded).squeeze(-1)
        return self.output(features).squeeze(1)


def make_vocab(texts: list[str], max_words: int = 12000) -> dict[str, int]:
    counter = Counter()
    for text in texts:
        counter.update(tokenize(text))
    vocab = {"<pad>": 0, "<unk>": 1}
    for word, _ in counter.most_common(max_words - 2):
        vocab[word] = len(vocab)
    return vocab


def train_binary(model: nn.Module, train_loader: DataLoader, val_loader: DataLoader, epochs: int = 4) ->
dict[str, list[
    model.to(DEVICE)
    criterion = nn.BCEWithLogitsLoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    history = {"loss": [], "val_loss": [], "acc": [], "val_acc": []}
    for _ in range(epochs):
        model.train()
        total_loss = correct = total = 0
        for x, y in train_loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            optimizer.zero_grad()
            logits = model(x)
            loss = criterion(logits, y)
```

```
            loss.backward()
            optimizer.step()
            total_loss += float(loss.item()) * len(x)
            correct += int(((torch.sigmoid(logits) >= 0.5).float() == y).sum().item())
            total += len(x)
        val_loss, val_acc = evaluate_binary(model, val_loader, criterion)
        history["loss"].append(total_loss / total)
        history["acc"].append(correct / total)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
    return history


def evaluate_binary(model: nn.Module, loader: DataLoader, criterion: nn.Module) -> tuple[float, float]:
    model.eval()
    total_loss = correct = total = 0
    with torch.no_grad():
        for x, y in loader:
            x, y = x.to(DEVICE), y.to(DEVICE)
            logits = model(x)
            total_loss += float(criterion(logits, y).item()) * len(x)
            correct += int(((torch.sigmoid(logits) >= 0.5).float() == y).sum().item())
            total += len(x)
    return total_loss / total, correct / total
```